QueryMind AI

End-to-End Test Plan

Text-to-SQL Analytics Engine

Version	1.0
Date	May 11, 2026
Stack	FastAPI · Next.js · PostgreSQL · Qdrant · Docling · OpenAI
Total TCs	47

1. Overview

This document defines the end-to-end test plan for QueryMind AI, a Text-to-SQL analytics engine. It covers all layers of the application from infrastructure health checks through to frontend interactions, including negative and edge-case scenarios.

1.1 Application Architecture

- Backend: FastAPI (Python 3.11+) on port 8000
- Frontend: Next.js 14 (TypeScript) on port 3000
- Relational DB: PostgreSQL 16 on port 5432
- Vector DB: Qdrant on port 6333
- Document parser: Docling (PDF, DOCX, HTML, Markdown, TXT)
- LLM: OpenAI GPT-4o via API

1.2 Query Pipeline (7 steps)

1. Classifier — detect intent (analytical / invalid / ambiguous)
2. Schema Linker — map entities to DB columns
3. Examples Retriever — fetch top-k few-shot pairs from Qdrant
4. Generator — OpenAI produces SQL
5. Validator — guardrails + syntax check
6. Executor — run SQL against PostgreSQL, cap at max_rows
7. Formatter — table output + natural-language summary

1.3 Scope

- In scope: all REST API endpoints, admin panels, query pipeline, document ingestion, frontend UI, negative tests
- Out of scope: load/stress testing, security penetration testing, browser compatibility beyond Chrome

2. Test Environment

2.1 Prerequisites

- Docker Desktop running
- Python 3.11+ with uv package manager
- Node.js 20+
- Valid OPENAI_API_KEY set in backend/.env
- DBeaver or psql for direct DB inspection

2.2 Startup Sequence

8. docker compose up -d (starts PostgreSQL + Qdrant)
9. cd backend && uv run uvicorn app.main:app --reload --port 8000
10. cd frontend && npm run dev
11. Verify: curl http://localhost:8000/health returns {status: ok}
12. Verify: http://localhost:6333/dashboard loads Qdrant UI

2.3 Test Data

- 1,000 customers | 500 products | 20 suppliers | 7 categories
- 3,000 orders | order_items | 2,000 reviews
- Seeded via: uv run python scripts/seed_db.py

3. Infrastructure Tests

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-001	**Postgres health**	Run: docker compose psCheck postgres STATUS column	Status = healthy, port 5432 mapped	—	
TC-002	**Qdrant health**	GET http://localhost:6333/healthz	Response body: 'healthz check passed'	—	
TC-003	**Backend health**	GET http://localhost:8000/health	{"status":"ok","service":"QueryMind AI"}	—	
TC-004	**Frontend loads**	Open http://localhost:3000 in browser	Page renders without console errors, sidebar visible	—	
TC-005	**Qdrant dashboard**	Open http://localhost:6333/dashboard	Qdrant Web UI loads, collections panel visible	—	
TC-006	**API docs**	Open http://localhost:8000/docs	Swagger UI loads, all routers listed	—	

4. Schema Manager Tests

Base URL: /admin/schema

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-010	**Introspect live schema**	GET /admin/schema/introspect	Returns JSON map of all tables and columns from PostgreSQL	—	
TC-011	**List columns (empty)**	GET /admin/schema/columns (before adding any)	Returns empty array []	—	
TC-012	**Create column description**	POST /admin/schema/columnsBody: {"table_name":"orders","column_name":"status","description":"Order lifecycle status","synonyms":["state","phase"],"is_sensitive":false}	HTTP 201, returns object with id, created_at	—	
TC-013	**List columns after create**	GET /admin/schema/columns	Array contains the record created in TC-012	—	
TC-014	**Update column description**	PATCH /admin/schema/columns/{id} Body: {"description": "Updated description"}	HTTP 200, description field reflects new value	—	
TC-015	**Delete column**	DELETE /admin/schema/columns/{id}	HTTP 204, subsequent GET /columns does not include the record	—	
TC-016	**Delete non-existent column**	DELETE /admin/schema/columns/99999	HTTP 404 with detail: Column not found	—	

5. Examples Manager Tests

Base URL: /admin/examples — examples are mirrored to Qdrant collection: few_shot_examples

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-020	**List examples (empty)**	GET /admin/examples	Returns [] before any examples added	—	
TC-021	**Create example**	POST /admin/examplesBody: {"question":"How many orders were placed last month?","sql":"SELECT COUNT(*) FROM orders WHERE created_at >= NOW() - INTERVAL '1 month'","query_type":"analytical"}	HTTP 201, id assigned, verify in Qdrant dashboard: collection few_shot_examples has 1 point	—	
TC-022	**Create 3 more examples**	POST /admin/examples x3 with different questions/SQL	All 4 visible via GET /admin/examples and in Qdrant	—	
TC-023	**Filter by query_type**	GET /admin/examples?query_type=analytical	Only examples with query_type=analytical returned	—	
TC-024	**Update example**	PATCH /admin/examples/{id}Body: {"question": "Updated question text"}	HTTP 200, Qdrant point payload updated (verify via /dashboard)	—	
TC-025	**Delete example**	DELETE /admin/examples/{id}	HTTP 204, point removed from Qdrant collection	—	
TC-026	**Sync embeddings**	POST /admin/examples/sync-embeddings	HTTP 202, {message: Sync complete}, Qdrant collection matches DB count	—	

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-027	**Delete non-existent example**	DELETE /admin/examples/99999	HTTP 404 with detail: Example not found	—	

6. Guardrails Config Tests

Base URL: /admin/guardrails — default keys: allow_select, allow_insert, allow_update, allow_delete, max_rows, restrict_to_schema

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-030	**List guardrails**	GET /admin/guardrails	Returns 6 default guardrail records with keys and values	—	
TC-031	**Verify defaults**	GET /admin/guardrails, inspect values	allow_select=true, allow_insert=false, allow_update=false, allow_delete=false, max_rows=500	—	
TC-032	**Update max_rows**	PATCH /admin/guardrails/max_rows Body: {"value":"100"}	HTTP 200, value=100 returned	—	
TC-033	**Enable INSERT guardrail**	PATCH /admin/guardrails/allow_insertBody: {"value":"true"}	HTTP 200, allow_insert=true	—	
TC-034	**Non-existent guardrail key**	PATCH /admin/guardrails/invalid_keyBody: {"value":"x"}	HTTP 404 with detail: Guardrail key not found	—	
TC-035	**Restore defaults after testing**	PATCH max_rows back to 500, allow_insert back to false	All guardrails restored to original values	—	

7. Model Config Tests

Base URL: /admin/config — default keys: model, temperature, max_tokens, dialect

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-036	**List model config**	GET /admin/config	Returns 4 records: model, temperature, max_tokens, dialect	—	
TC-037	**Verify defaults**	Inspect GET /admin/config values	model=gpt-4o, temperature=0.0, max_tokens=1000, dialect=postgresql	—	
TC-038	**Update temperature**	PATCH /admin/config/temperatureBody: {"value":"0.2"}	HTTP 200, value=0.2	—	
TC-039	**Update max_tokens**	PATCH /admin/config/max_tokensBody: {"value":"500"}	HTTP 200, value=500	—	
TC-040	**Invalid config key**	PATCH /admin/config/nonexistentBody: {"value":"x"}	HTTP 404 with detail: Config key not found	—	
TC-041	**Restore defaults**	PATCH temperature to 0.0, max_tokens to 1000	Config back to original values	—	

8. Query Logs Tests

Base URL: /admin/logs — run several queries first via POST /query to populate logs

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-042	**List logs (paginated)**	GET /admin/logs?page=1&page_size=20	Returns {items, total, page, page_size} structure	—	
TC-043	**Filter by status=success**	GET /admin/logs?status=success	Only logs with status=success returned	—	
TC-044	**Filter by status=error**	GET /admin/logs?status=error	Only logs with status=error returned	—	
TC-045	**Pagination — page 2**	GET /admin/logs?page=2&page_size=5	Returns next 5 results, page=2 in response	—	
TC-046	**Log fields**	Inspect a single log entry	Fields present: nl_query, query_type, generated_sql, status, latency_ms, row_count, created_at	—	

9. Document Ingestion Tests (Docling + Qdrant)

Base URL: /admin/documents — Docling parses files, chunks are stored in Qdrant collection: schema_docs

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-047	**Upload PDF**	POST /admin/documents/uploadForm-data: file=schema_docs.pdf	HTTP 202, {doc_name, chunks_indexed > 0}, collection schema_docs visible in Qdrant	—	*Prepare a small PDF with table descriptions*
TC-048	**Upload DOCX**	POST /admin/documents/uploadForm-data: file=data_dictionary.docx	HTTP 202, chunks indexed into schema_docs	—	
TC-049	**Upload Markdown**	POST /admin/documents/uploadForm-data: file=README.md	HTTP 202, chunks indexed	—	
TC-050	**Unsupported file type**	POST /admin/documents/uploadForm-data: file=data.csv	HTTP 422 — Unsupported file type .csv	—	
TC-051	**Semantic search**	GET /admin/documents/search?q=customer+segments&top_k=3	Returns top-3 relevant chunks with doc_name and score	—	*Run after uploading docs*
TC-052	**Delete document**	DELETE /admin/documents/schema_docs.pdf	HTTP 204, chunks removed from Qdrant	—	
TC-053	**Search after delete**	GET /admin/documents/search?q=customer+segments	Results no longer include chunks from deleted doc	—	

10. Query Pipeline Tests (Text-to-SQL)

POST /query — Body: {question: string} — requires valid OPENAI_API_KEY

10.1 Analytical Queries

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-054	**Simple aggregate**	"How many customers do we have?"	Returns COUNT, SQL uses SELECT COUNT(*) FROM customers, status=success	—	
TC-055	**Top-N query**	"What are the top 5 products by revenue?"	Returns table with product name + revenue, ORDER BY DESC LIMIT 5	—	
TC-056	**Date filter**	"How many orders were placed last month?"	SQL contains date filter using NOW() - INTERVAL, row_count >= 0	—	
TC-057	**Multi-table JOIN**	"Show me the average order value by customer segment"	SQL JOINs orders and customers, GROUP BY segment	—	
TC-058	**Trend query**	"What is the month-over-month revenue trend for 2024?"	SQL groups by month, returns time-series data	—	
TC-059	**Supplier query**	"Which suppliers have the most out-of-stock products?"	SQL JOINs products and suppliers, filters stock_quantity=0	—	
TC-060	**Review sentiment**	"What is the average product rating by category?"	SQL JOINs reviews, products, categories,	—	

TC #	Test Case	Steps	Expected Result	Status	Notes
			AVG(rating) GROUP BY category		

10.2 Guardrail Enforcement

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-061	**SELECT allowed by default**	"List all customers"	Query executes, rows returned, status=success	—	
TC-062	**INSERT blocked**	"Add a new customer named John"	status=error or guardrail rejection — INSERT not allowed	—	
TC-063	**DELETE blocked**	"Delete all cancelled orders"	status=error or guardrail rejection — DELETE not allowed	—	
TC-064	**max_rows enforced**	Set max_rows=10 via guardrails, run query that returns many rows	Result set capped at 10 rows	—	*Restore max_rows =500 after*

10.3 Edge Cases

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-065	**Ambiguous query**	"Show me data"	Query classified as ambiguous, meaningful error or clarification returned	—	
TC-066	**Non-SQL question**	"What is the capital of France?"	Classifier rejects as invalid/off-topic, no SQL generated	—	

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-067	**Empty question**	POST /query Body: {"question":""}	HTTP 422 validation error or meaningful error message	—	
TC-068	**SQL injection attempt**	"Show all users; DROP TABLE customers;--"	Validator blocks, status=error, table intact in DB after	—	*Verify via DBeaver*
TC-069	**Very long question**	Send a question with 2000+ characters	Handled gracefully — either processes or returns error, no 500	—	
TC-070	**Query with no results**	"List orders placed in 1990"	Returns empty result set, status=success, row_count=0	—	

11. Frontend UI Tests

Browser: Chrome (latest). Base URL: http://localhost:3000

TC #	Test Case	Steps	Expected Result	Status	Notes
TC-071	**Home page loads**	Navigate to http://localhost:3000	Page renders, sidebar shows navigation links	—	
TC-072	**Query page — submit question**	Navigate to /query, type "How many customers do we have?", press Send	Result table appears below input, SQL visible in response	—	
TC-073	**Schema Manager UI**	Navigate to /admin/schema, add a column description via form	New row appears in table on page	—	
TC-074	**Examples Manager UI**	Navigate to /admin/examples, create a new example	Example appears in list, verify Qdrant count increases	—	
TC-075	**Query Logs UI**	Navigate to /admin/logs after running several queries	Log table populates with nl_query, status, latency columns	—	
TC-076	**Guardrails UI**	Navigate to /admin/guardrails	All 6 guardrail toggles/values displayed	—	
TC-077	**Model Config UI**	Navigate to /admin/config	All 4 config keys shown with editable values	—	
TC-078	**API error displayed**	Submit query with invalid OPENAI_API_KEY	Frontend shows error message gracefully — no blank screen	—	

12. Test Execution Summary

Module	Total TCs	Pass	Fail	Skipped	Pass Rate
Infrastructure	6				
Schema Manager	7				
Examples Manager	8				
Guardrails Config	6				
Model Config	6				
Query Logs	5				
Document Ingestion	7				
Query Pipeline	17				
Frontend UI	8				
TOTAL	**70**				

12.1 Sign-off

Role	Name	Date
Tester		
Reviewer		
Approver		

12.2 Defect Log

Bug #	TC Ref	Title	Steps to Reproduce	Severity	Status	Owner